



GOKSTAD AKADEMIET

Backend-programmering, 1. år, Oslo

Emne 2 -- Operasjonell Backend-programmering

Faglærer: Ali Zedan

Arbeidskrav 2

Datoer:	6. mars 2026 -- 20. mars 2026
Tillatte ressurser:	• Alle • Bruk av AI er tillatt, se merknad
Innlevering:	Komplett C#-løsning
Vurdering:	godkjent / ikke godkjent

Hotellbookingsystem -- Kravspesifikasjon

Innledning

I Arbeidskrav 1 bygde du et biblioteksystem med fokus på arv, abstraksjon og innkapsling. I dette arbeidskravet skal du videreutvikle disse ferdighetene ved å bygge et **hotellbookingsystem**. Systemet skal i tillegg demonstrere bruk av **interfaces** og **polymorfi**, og du skal levere et enkelt **UML-klassediagram** som viser systemets struktur.

Systemet skal håndtere romreservasjoner, gjester med ulike privilegier og betalingsmetoder - alt gjennom et konsollbasert grensesnitt.

Oversikt

Bygg en konsollapplikasjon som simulerer et hotellbookingsystem med:

- Ulike romtyper (enkeltrom, dobbeltrom, suite)
- Gjester med ulike roller (vanlig gjest, VIP-gjest)
- Bookingsystem med inn- og utsjekk
- Betalingsmetoder implementert via interface
- Grunnleggende tester for kjernefunksjonalitet
- Et UML-klassediagram over systemets klasser og relasjoner

Det primære formålet med denne oppgaven er å vurdere din evne til å anvende objektorienterte prinsipper i en praktisk kontekst, med særlig fokus på interfaces, polymorfi og testing.

Funksjonelle krav

1. Romklasser

Lag en abstrakt baseklasse **Room** med følgende fellesegenskaper:

- **RoomNumber** (string) -- unik identifikator, f.eks. "101"
- **RoomType** (string) -- settes av subklassen
- **PricePerNight** (decimal) -- pris i NOK, må være positiv
- **IsAvailable** (bool) -- om rommet er ledig for booking
- **MaxGuests** (int) -- maksimalt antall gjester

Abstrakt metode: **DisplayRoomInfo()** -- må implementeres av alle arvede klasser for å vise romspesifikk informasjon.

Lag følgende konkrete klasser som arver fra Room:

- **SingleRoom** -- HasDesk (bool), MaxGuests = 1, standard PricePerNight = 800
- **DoubleRoom** -- HasExtraBed (bool), MaxGuests = 2, standard PricePerNight = 1200
- **Suite** -- HasJacuzzi (bool), HasLounge (bool), MaxGuests = 4, standard PricePerNight = 3500

Hver klasse må implementere DisplayRoomInfo() for å vise relevant informasjon om rommet.

2. Gjesteklasser

Lag en abstrakt baseklasse **Guest** med:

- **GuestId** (string) -- autogenerated, unik (f.eks. "G001")
- **Name** (string)
- **Email** (string) -- må valideres (skal inneholde '@')
- **ActiveBookings** (List<Booking>) -- liste over gjestens aktive bookinger

Abstrakt metode: **GetDiscount(decimal basePrice)** -- returnerer endelig pris etter eventuell rabatt.

Lag følgende konkrete gjesteklasser:

- **RegularGuest** -- Ingen rabatt. Kan ha maks 3 aktive bookinger samtidig.
- **VipGuest** -- Får 15% rabatt på alle rom. Kan ha maks 10 aktive bookinger. Har i tillegg en egenskap **LoyaltyPoints** (int) som øker med 10 for hver booking.

Implementer GetDiscount() basert på gjestens type.

3. Interface: IPayable

Definer et interface **IPayable** som alle betalingsmetoder må implementere:

- **bool ProcessPayment(decimal amount)** -- simulerer betaling, returnerer true

- string GetPaymentInfo() -- returnerer en beskrivelse av betalingsmetoden

Implementer følgende betalingsklasser som bruker interfacet:

- **CardPayment** -- egenskaper: CardNumber (string, kun siste 4 siffer vises), CardType (string, f.eks. "Visa"). ProcessPayment() returnerer alltid true (simulert).
- **VippsPayment** -- egenskaper: PhoneNumber (string). ProcessPayment() returnerer alltid true (simulert).

Booking-klassen skal bruke IPayable-interfacet, ikke en konkret betalingsklasse.

4. Booking-klassen

Lag en klasse **Booking** med:

- BookingId (string) -- autogenerated, unik (f.eks. "BK001")
- Room (Room) -- referanse til det bookedde rommet
- Guest (Guest) -- referanse til gjesten
- CheckInDate (DateTime)
- CheckOutDate (DateTime)
- PaymentMethod (IPayable) -- bruker interfacet
- IsPaid (bool) -- settes etter vellykket betaling

Metoder:

- CalculateTotalPrice() -- beregner antall netter × pris per natt, kaller GetDiscount() på gjesten for endelig pris
- CheckIn() -- setter rommet som opptatt (IsAvailable = false)
- CheckOut() -- setter rommet som ledig igjen og logger utsjekk

5. Hotel-klassen

Lag en klasse **Hotel** som håndterer:

- Name (string) -- hotellets navn
- Rooms (List<Room>) -- alle rom i systemet
- Guests (List<Guest>) -- alle registrerte gjester
- BookingHistory (List<Booking>) -- logg over alle bookinger

Metoder:

- RegisterGuest(Guest guest) -- legger til ny gjest, valider at e-post ikke allerede er registrert
- GetAvailableRooms(DateTime checkIn, DateTime checkOut) -- viser alle ledige rom i perioden
- CreateBooking(string guestId, string roomNumber, DateTime checkIn, DateTime checkOut, IPayable payment) -- sjekker at gjesten kan booke, at rommet er ledig, oppretter booking og gjennomfører betaling
- CancelBooking(string bookingId) -- kansellerer booking og setter rommet som ledig igjen

- `GetGuestBookings(string guestId)` -- viser alle aktive bookinger med totalpriser

Systemet skal håndtere feil på en god måte, for eksempel hvis gjesten prøver å booke et opptatt rom, eller hvis en `RegularGuest` allerede har 3 aktive bookinger.

UML-klassediagram

Lever et **UML-klassediagram** som viser klassene i systemet ditt og relasjonene mellom dem. Diagrammet skal inkludere:

- Alle klasser med de viktigste feltene og metodene
- Arveforhold (pil med tom trekant)
- Interface-implementering (stiplet pil med tom trekant)
- Assosiasjoner mellom klasser (f.eks. `Booking` "har en" `Room` og "har en" `Guest`)

Diagrammet kan tegnes for hånd og fotograferes, eller lages digitalt med et verktøy som `draw.io` (gratis på `diagrams.net`). Lever det som bilde (.png/.jpg) eller PDF vedlagt i zip-filen.

Testing

Skriv **minst 4 enkle tester** som verifiserer at systemet oppfører seg korrekt. Du kan skrive testene direkte i konsollprosjektet -- ingen eksternt testrammeverk kreves:

- **Test 1:** `VipGuest.GetDiscount()` returnerer korrekt pris med 15% rabatt. Eksempel: grunnpris 1000 kr skal gi 850 kr.
- **Test 2:** `Booking.CalculateTotalPrice()` beregner riktig totalpris for et gitt antall netter og romtype.
- **Test 3:** `Hotel.CreateBooking()` gir feilmelding hvis rommet allerede er opptatt. • **Test 4:** `CheckOut()` setter rommet som ledig (`IsAvailable = true`) etter utsjekk.
- **Bonus:** Test at `RegularGuest` ikke kan ha mer enn 3 aktive bookinger.

Testene skal kjøres automatisk når programmet starter og skrive ut `PASSED` eller `FAILED` i konsollen.

Informasjonssikkerhetskrav

Systemet skal ivareta grunnleggende informasjonssikkerhet:

- **Tilgangskontroll:** Kun registrerte gjester kan opprette bookinger
- **Dataintegritet:** `RoomNumber`, `GuestId` og `BookingId` skal være unike og autogenerated (f.eks. "G001", "BK001")
- **Innkapsling:** Viktige felter (som `IsAvailable`, `IDer`) skal være private med kontrollert tilgang via properties
- **Validering:** Sjekk at e-postadresser inneholder '@', at priser er positive, og at `CheckInDate` er før `CheckOutDate`
- **Logging:** Alle booking- og utsjekk-operasjoner skal logges i `BookingHistory`

Implementer disse kravene ved å bruke riktige tilgangsmodifikatorer (private, public, protected) og properties.

Objektorienterte programmeringskrav

Din løsning må demonstrere følgende OOP-konsepter:

1. Innkapsling

- Bruk private felter med public properties
- Valider data hvor det er hensiktsmessig (f.eks. PricePerNight > 0, gyldig Email)
- Bruk auto-properties der det passer

2. Arv

- Opprett abstrakte baseklasser (Room, Guest)
- Implementer konkrete arvede klasser
- Bruk 'abstract' nøkkelordet for abstrakte metoder
- Bruk 'override' nøkkelordet i arvede klasser

3. Abstraksjon

- Definer abstrakte metoder i baseklassene (DisplayRoomInfo(), GetDiscount())
- La hver konkret klasse implementere sine egne versjoner
- Skjul implementeringsdetaljer fra andre klasser

4. Interface

- Definer og bruk IPayable-interfacet
- Implementer interfacet i CardPayment og VippsPayment
- Booking-klassen bruker IPayable, ikke konkrete betalingsklasser

5. Polymorfi

- Bruk baseklassereferanser (List<Room>, List<Guest>)
- Demonstrer at forskjellige objekttyper kan håndteres via felles baseklasse eller interface

Retningslinjer for kodekvalitet

Kodekompleksitet

- Hold metoder enkle: Hver metode skal håndtere én spesifikk oppgave
- Unngå nestet logikk: For mange nestede if-else kan gjøre koden vanskelig å lese
- Bruk collections effektivt: Bruk List<T> for å håndtere rom, gjester og bookinger
- Validering: Sørg for input-validering (f.eks. gyldige menyvalg, e-postformat)
- Unngå hardkoding: Definer konstanter eller konfigurerbare verdier

Kodekonvensjoner

- Navnekonvensjoner: Bruk PascalCase for klasser og metoder, camelCase for privatefelter
- Kommentering: Alle public metoder og properties skal ha kommentarer
- Klassestruktur: Organiser koden logisk med én klasse per fil

Eksempel på konsolloutput

```
=== Hotel Gokstad ===
1. Show available rooms
2. Create booking
3. Check in
4. Check out
5. Show my bookings
6. Register new guest
0. Exit

Choose option: 1
Enter check-in date (dd.MM.yyyy): 10.03.2026
Enter check-out date (dd.MM.yyyy): 13.03.2026

=== Available rooms (10.03 - 13.03) ===
[101] SingleRoom - 800 kr/night - Max 1 guest - HasDesk: Yes
[201] DoubleRoom - 1200 kr/night - Max 2 guests - HasExtraBed: No
[301] Suite      - 3500 kr/night - Max 4 guests - HasJacuzzi: Yes | HasLounge: Yes

Choose option: 2
Enter GuestId: G001
Enter RoomNumber: 201
Payment method (1=Card, 2=Vipps): 1
Card number (last 4 digits): 4242

✓ Booking BK001 created for Kari Nordmann
  DoubleRoom 201 -- 10.03.2026 to 13.03.2026 (3 nights)
  Total price: 3600 kr
  Payment approved with Visa **** 4242
```

Begrensninger

- Kun konsollapplikasjon -- Ingen GUI kreves
- Ingen database -- Data lagres i minnet (List<T>)
- Valgfritt: File I/O -- Du kan implementere lagring til fil hvis du ønsker (ikke påkrevd)

Innleveringskrav

README.md

Inkluder en grunnleggende README.md-fil med løsningen din.

Som utgangspunkt, se makeareadme.com

Dette er et ideelt sted å inkludere eventuelle AI-prompts som ble brukt.

UML-diagram

Lever UML-klassediagrammet som bilde (.png/.jpg) eller PDF i zip-filen.

Versjonskontroll

Bruk av Git for versjonskontroll er valgfritt, men anbefales. Å spore endringene dine med commits demonstrerer god utviklingspraksis.

Innlevering

- Ferdig løsning og støttedokumenter skal leveres på MinGA (Moodle)
- Lever kun én .zip-fil
- Hvis innleveringen din ikke kan åpnes, kan den ikke vurderes
- Forsikre deg om at det ikke er feilplasserte, manglende eller ufullstendige filer

Det er ikke sensoren sin jobb å reparere løsningen din.

Vurdering

Arbeidskravet vurderes som godkjent eller ikke godkjent.

Bruk av AI

Bruk av AI er tillatt.

Du må inkludere alle prompts som ble brukt som en del av dokumentasjonen. Du kan gjøre dette enten som en del av README.md-filen, eller som en separat fil (.md/.txt/.doc(x)).

Alternativt kan du oppgi at du ikke brukte AI i løsningen din. (Å lyve om dette regnes som plagiat, og det vil ikke bli tolerert.)

Vedlegg A: Risikovurdering og prosjektstyring

Som en del av læringsutbyttet i emnet, er det viktig å reflektere over potensielle risikoer i utviklingsprosjekter og hvordan de kan håndteres. Selv om dette arbeidskravet er individuelt og relativt avgrenset, kan du vurdere følgende:

Vanlige risikoer i programvareutvikling:

- **Tidspress:** Planlegg arbeidet ditt over de to ukene, ikke vent til siste øyeblikk
- **Misforståtte krav:** Les kravspesifikasjonen nøye, still spørsmål tidlig
- **Tekniske utfordringer:** Bruk oppgavehjelp torsdager, søk hjelp når du står fast
- **Scope creep:** Fokuser på kjernekravene først, legg til ekstra funksjonalitet etterpå

Tilnærming til prosjektstyring:

1. Bryt ned oppgaven: Del arbeidet inn i mindre deler (f.eks. først Room-klasser, deretter Guest-klasser, etc.)
2. Test underveis: Ikke vent til slutten med å teste -- test hver klasse etter hvert som du lager dem
3. Commit ofte: Hvis du bruker Git, commit endringer regelmessig slik at du kan gå tilbake hvis noe går galt
4. Hold det enkelt først: Implementer en fungerende minimumsversjon før du legger til ekstra features

Ved å ta en strukturert tilnærming til arbeidet og være bevisst på potensielle fallgruver, øker du sjansene for å levere et vellykket arbeidskrav innen fristen.

Vedlegg B: Tips for menyimplementering

For dette arbeidskravet anbefaler jeg å holde menyimplementeringen enkel. Fokuset skal være på å demonstrere OOP-prinsipper, ikke på å bygge avanserte brukergrensesnitt.

Grunnleggende tilnærming

En enkel meny kan bygges ved hjelp av `Console.WriteLine()` for visning og `Console.ReadLine()` for input:

```
Console.WriteLine("Available operations:");
Console.WriteLine(" 1. Show rooms");
Console.WriteLine(" 2. Create booking");
Console.Write("Choose option (1-2): ");
string input = Console.ReadLine();
```

For validering fungerer en enkel løkke godt:

```
int GetValidChoice(int min, int max)
{
    while (true)
```

```
        {           if (int.TryParse(Console.ReadLine(), out int
choice)           && choice >= min && choice <= max)
        {           return choice;           }
        Console.Write("Invalid choice. Try again: ");
        }
    }
```

Denne tilnærmingen er tilstrekkelig for dette arbeidskravet og holder fokuset på klassedesign og OOP-implementering.

Lykke til med arbeidskravet!